6.100B: Introduction to Computational Thinking and Data Science

JOHN GUTTAG

MIT EECS

Lecture Being Recorded for MIT OCW

tell me about MIT OCW

MIT OpenCourseWare (OCW) is one of those quietly world-changing projects that MIT just… put on the internet and walked away from. In the best possible way.

🎓 What MIT OpenCourseWare Is

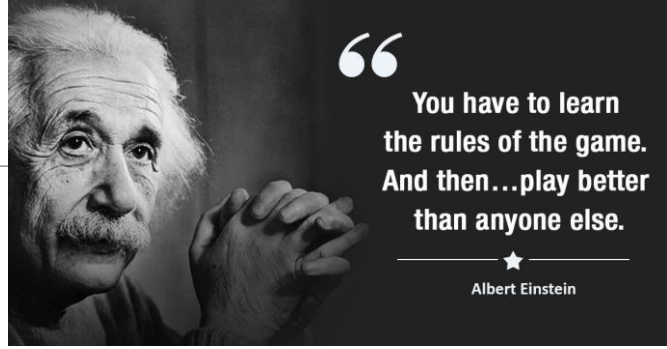
MIT OCW is a **free, open-access repository of real MIT course materials**—made public starting in **2001**. No enrollment, no login, no tuition, no credit. Just the content.

Website: **ocw.mit.edu**

It's used by:

- Self-learners around the world
- University instructors cribbing syllabi and problem sets
- Students who want a *second explanation* from a different instructor
- Professionals refreshing or deepening technical knowledge

Welcome



- Lecture materials available on course website
  - Might change after lecture
- After class, make sure:
  - You have access to our course website
  - You have signed up for our course Piazza
  - You have installed Python via Pset 0 (not graded)
  - You complete Finger exercise 1
  - You are able to complete Microquiz 0 (not graded)
  - You understand course policies and schedule!

Speaking of Schedule

- It's unusual
- A six-unit subject stretched over most of term
  - To avoid interfering with closing rush of other subjects
    - Last class/quiz April 29
    - Last problem set due May 6
- Periods of introducing new material (lectures)
  - With no problem set to work on
- Periods of assimilating material (problem sets)
  - With no lectures to attend
  - Optional recitations
  - Occasional short quizzes

Collaboration Policy

- Okay for problem sets and finger exercises
  - Discussing overall approach with others
  - Getting help debugging
  - Using Python tutor
- Not okay for problem sets
  - Looking at code of others or from previous terms
  - Providing your code to others
  - Side-by-side coding
- Micro quizzes
  - All work must be yours
  - **No access to online materials of any sort**

Workload and Grading

- Finger exercises: 5%
- Five Problem sets: 35%
  But must total 400 to get A, and 350 to get B
- Five 35-minute quizzes: 60%
  - Drop worse score
- Grade guidelines
  - A >=90 total average
    - With >= 340 on non-dropped quizzes
    - With >= 400 on problem sets
  - B >= 80
    - Must total >= 300 on non-dropped quizzes
    - Must total >= 350 on problem sets
  - C >= 70
  - D >= 60

Snow-Day Policy

- Office hours canceled
- If recitation day, recitation canceled
- If lecture day, lecture rescheduled to next class period on which either a recitation or no-class is planned
- If quiz day, quiz rescheduled to next class period on which either a recitation or no-class is planned

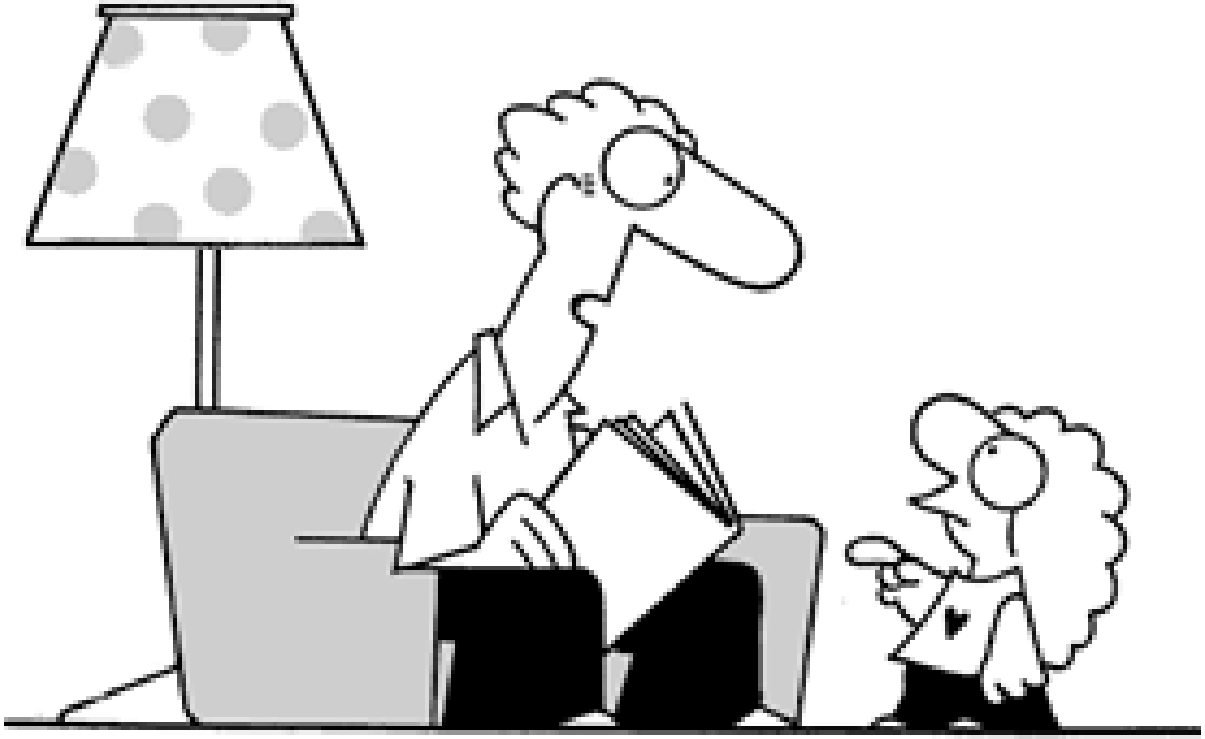
6.100B Prerequisites

- Experience writing object-oriented programs in Python 3
- Familiarity with concepts of computational complexity
- Familiarity with some basic algorithms
- 6.100A or 6.100L sufficient, but not necessary

How Does 6.100B Compare to 6.100A/L?

- Programming component of assignments perhaps a bit easier
- Focus more on the problem to be solved than on programming
- Lecture content is more abstract
  - More focus on methods, less on details
  - Includes things not needed for problem sets, but relevant to understanding foundational ideas, and for quizzes
- Lectures will be faster paced
  - Assumes that you use textbook and other written material as needed

In Case the Concept is Foreign

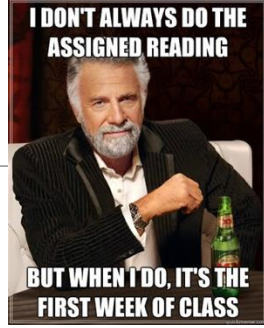


"It's called 'reading'. It's how people install new software into their brains."

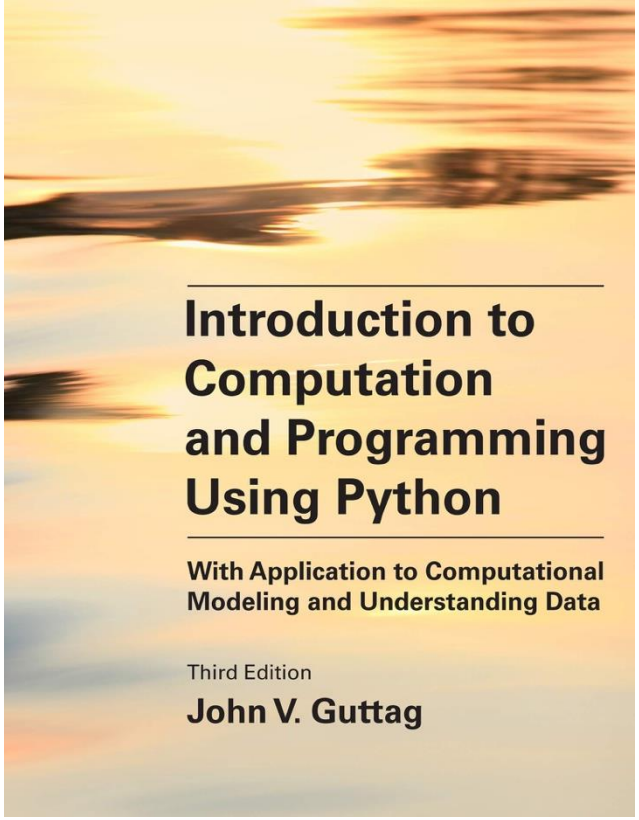
Topics

- "Computational Modeling" would be a better name for this subject
- Extensively revamped this year
  - Lots of new material in lectures
  - New problem sets
- Major topics
  - Using computation to **model** the world
    - Explanatory models
    - Predictive models
  - Extracting useful information from **data**
  - Becoming better and more efficient at **programming**
    - Mostly by practicing with problem sets
    - A few pointers during lectures

Reading for This Week



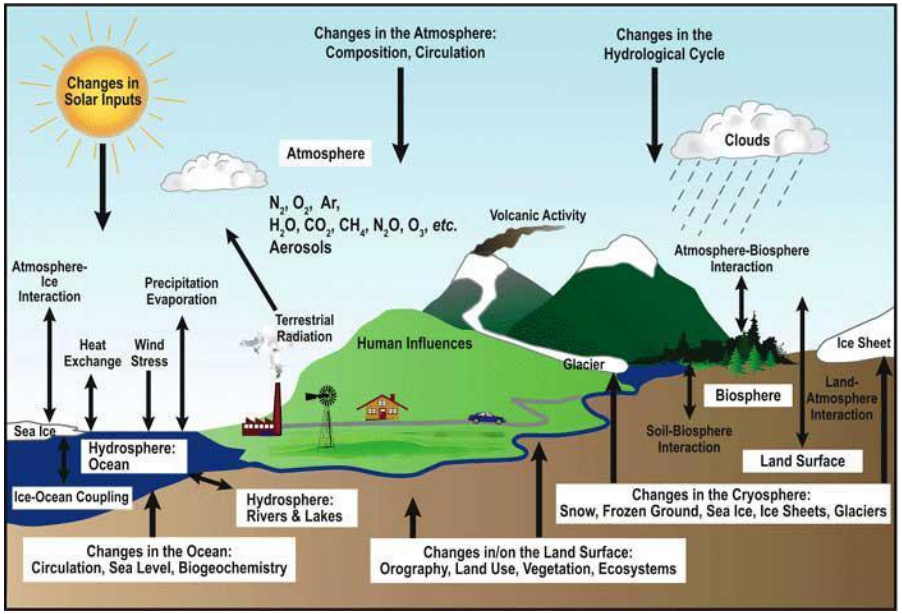
- Today
  - Section 14.1
- Next Lecture
  - Chapter 15

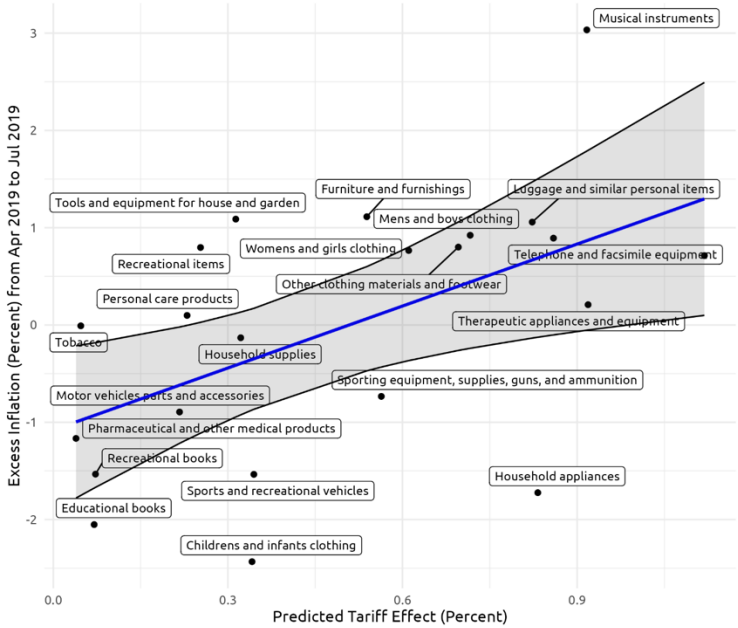


All code and an errata sheet at
https://github.com/guttag/Intro-to-Computation-and-Programming

Models

- Abstractions that help us to understand something that has happened or to predict the future
- Why has burning fossil fuels led to global climate change?
- What is the impact of increasing tariffs?



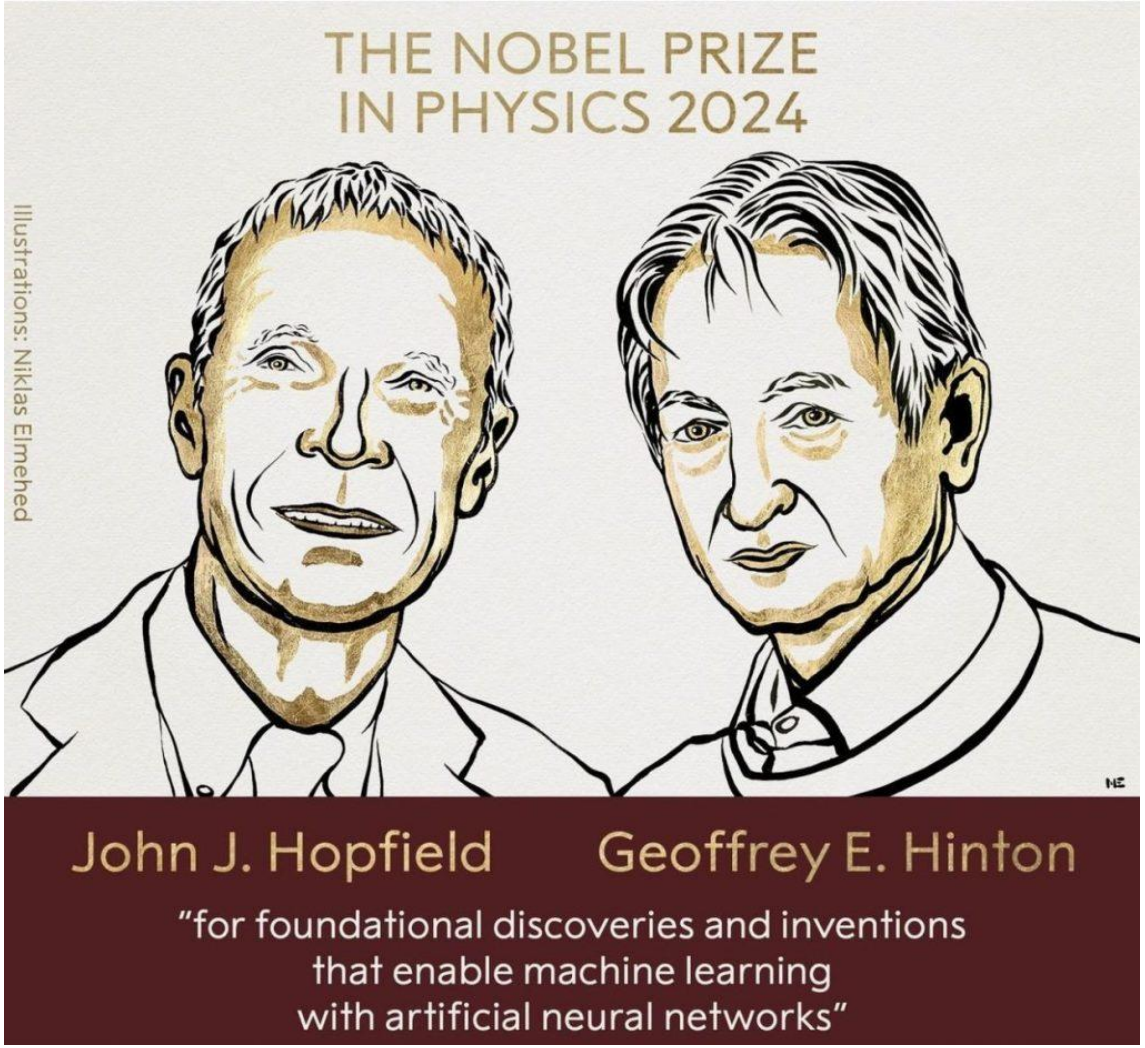


Computational Models

- Use computation to help understand the world:
  - Study behavior of complex physical system
    - Simulate solution to mathematical model that has no analytic solution
    - Measure statistical parameters from noisy data
  - Computational experiments can complement physical experiments and mathematical models (*in silico* vs. *in vitro* or *in vivo* vs. *in doctrina)*
- Computational experiments less expensive and easier to control

- Often can't run an actual experiment
  - Not easy to reduce fossil fuel consumption or change the tax code

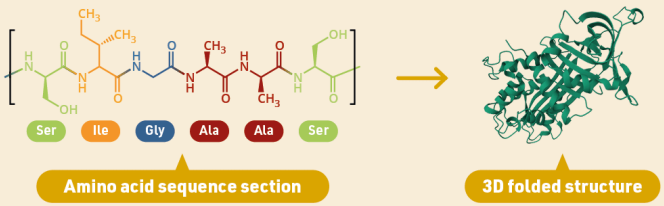
Computational Models and 2024 Physics Nobel



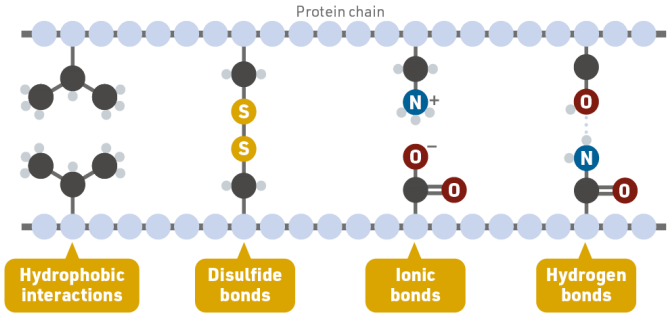
Computational Models and 2024 Chemistry Nobel

The 2024 Nobel Prize in Chemistry was awarded to **David Baker** for computational protein design and to **Demis Hassabis** and **John M. Jumper** for protein structure prediction.

Proteins are important biological molecules formed from 20 naturally occurring amino acids. Proteins form folded 3D structures which are key to their function and properties, but the exact way in which they fold is hard to predict. A protein with just 100 amino acids could have 10^{47} different 3D structures.

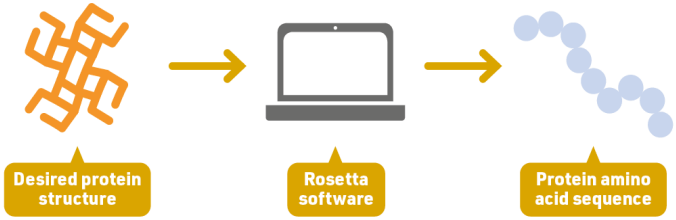


In 2020, **Demis Hassabis**, **John Jumper** and their co-workers unveiled an artificial intelligence model called AlphaFold2 to predict 3D folded structures of proteins. This is notoriously difficult because of the range of intermolecular forces in protein structures.



AlphaFold2 analyses amino acid sequences and evaluates how they might interact with each other. It has since been used to predict the structures of the almost 200 million known proteins.

David Baker developed Rosetta, software that also attempts to predict protein structures. He wondered if it was possible to work in the other direction: to start with a protein structure and use the software to work out its amino acid sequence.



Rosetta uses a database of protein structures and searches it for fragments with the same structure as the desired structure, pieces them together, then suggests an amino acid sequence based on this.

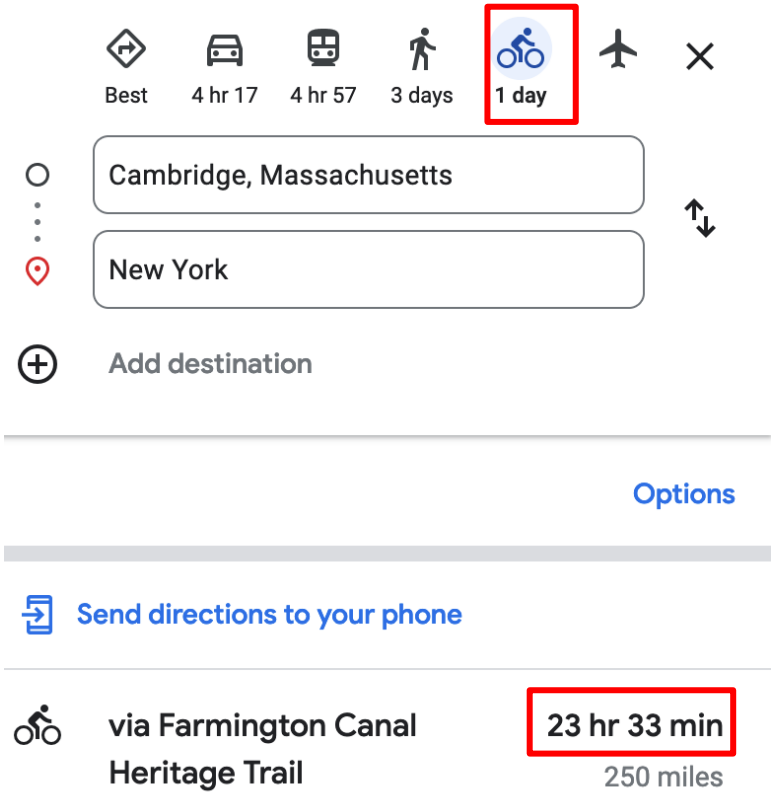
Baker's research group succeeded in doing this in 2003 to create an entirely new protein. They have since produced many other novel proteins that do not occur naturally.

Some Kinds of Models

- Optimization models
  - Find mathematical models that maximize (or minimize) some criterion, subject to constraints on solution
- Simulation models
  - Find models that explain observations from large numbers of random, noisy trials
- Statistical models
  - Find models that deduce parameters or label examples in presence of noise
  - Machine learning

What Is an Optimization Model?

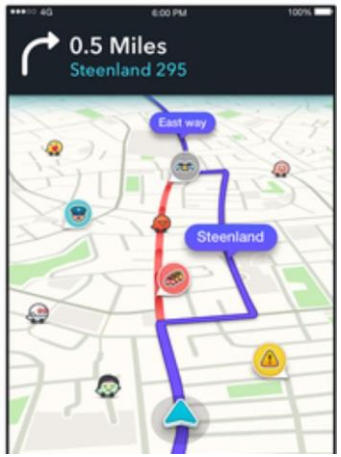
- An objective function that is to be maximized or minimized
  - E.g., minimize money spent traveling from Cambridge to NYC
- But perhaps merely optimizing objective function is not enough
- A set of constraints (possibly empty) that must be honored
  - E.g., expected transit time < 5 hours

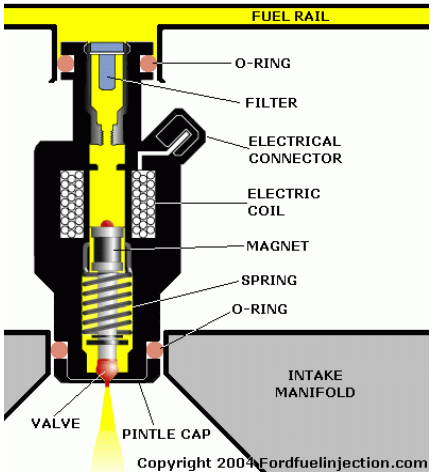


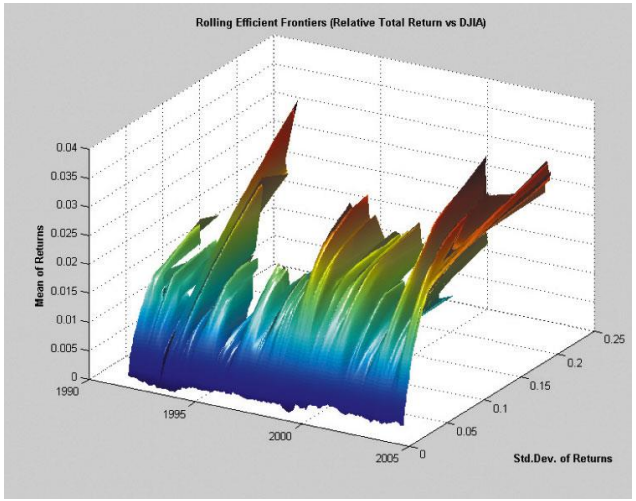
If you can average 10.6mph for 23.5 hours, congratulations

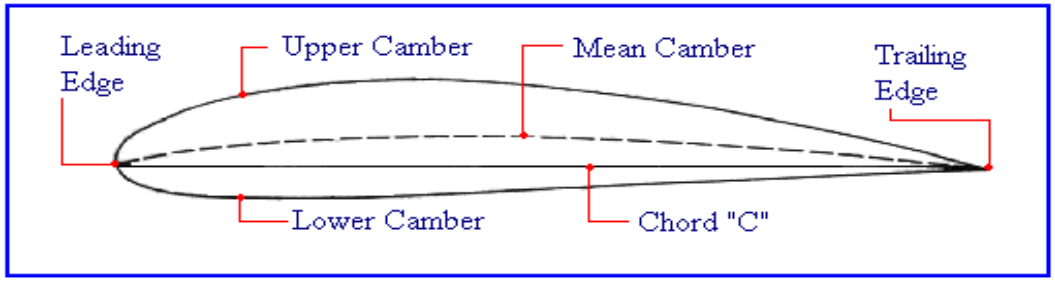
Optimization Problems

- Anytime you are trying to maximize or minimize something, you are solving an optimization problem









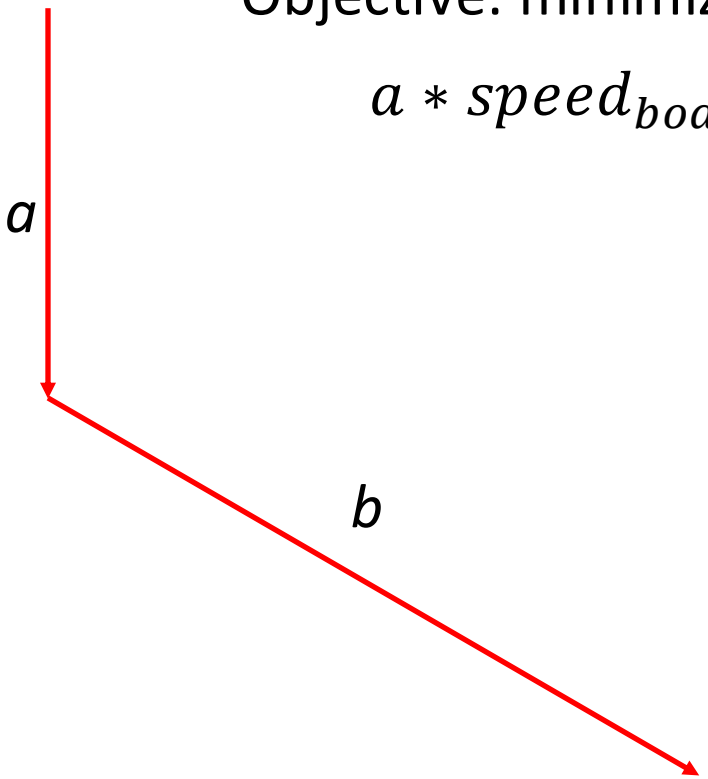
A Simple Optimization Problem

- A general needed to get his army to a fort due east of a river that ran from north to south.
- The river had a boat landing every mile.
- By boat, his army moved at boat_speed miles/hour. When going overland, the army moved at march_speed miles/hour.
- How far down river should the army travel by boat to minimize the time needed to reach the fort?

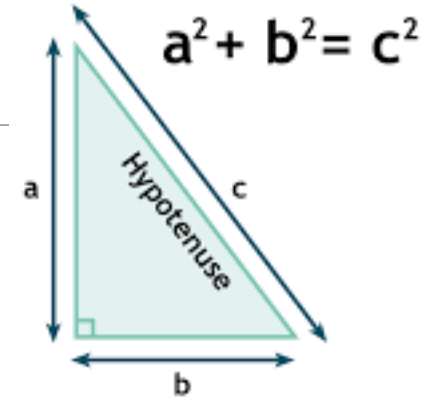
Let's Formalize It

Objective: minimize

$$a * speed_{boat} + b * speed_{march}$$



Let's Formalize It

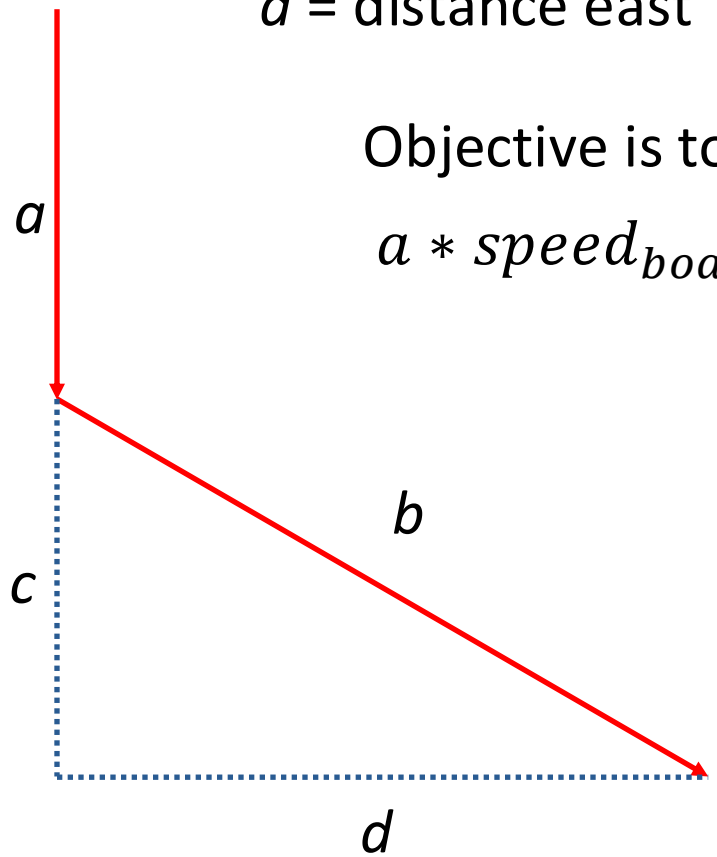


$a + c$ = north-south distance

d = distance east

Objective is to minimize

$$a * speed_{boat} + \boxed{\sqrt{c^2 + d^2}} * speed_{march}$$



Let's Find a Good-enough Solution

- Recall that there are a finite number of boat landings on the river
  - One every mile
- Use exhaustive enumeration
  - Compute the total elapsed time for each possible landing
  - Chose the landing with the minimal time

Let's Solve It

```
def find_landing(boat_speed, march_speed, dist_south, dist_east, plot = False):
    times = []
    incr = 1
    miles_south = 0

    while miles_south <= dist_south:
        dist_marched = ((dist_south-miles_south)**2+dist_east**2)**0.5
        tot_time = miles_south/boat_speed + dist_marched/march_speed
        times.append(tot_time)
        miles_south += incr

    if plot:
        plt.plot(times,
                 label = f'boat speed = {boat_speed},march_speed = {march_speed}')
        plt.title('Landing vs Time')
        plt.xlabel('Distance Down River')
        plt.ylabel('Total Time (hours)')
        plt.legend()
    best_idx = np.argmin(times)
    landing = best_idx*incr
    print(f'Best landing is {landing} miles south of start')
    dist_marched = ((dist_south-landing)**2+dist_east**2)**0.5
    print(f'Distance marched would be {dist_marched:.2f} miles')
    print(f'Time is {times[best_idx]:.2f} hours')
```

Stop and Think

- What should it print for `find_landing(1, 1, 100, 50)`

Let's Try It

- What should it print for `find_landing(1, 1, 100, 50)`

  Best landing is 0 miles south of start
  Distance marched would be 111.80 miles
  Time is 111.80 hours

- Let's try `find_landing(1.5, 1, 100, 50)`

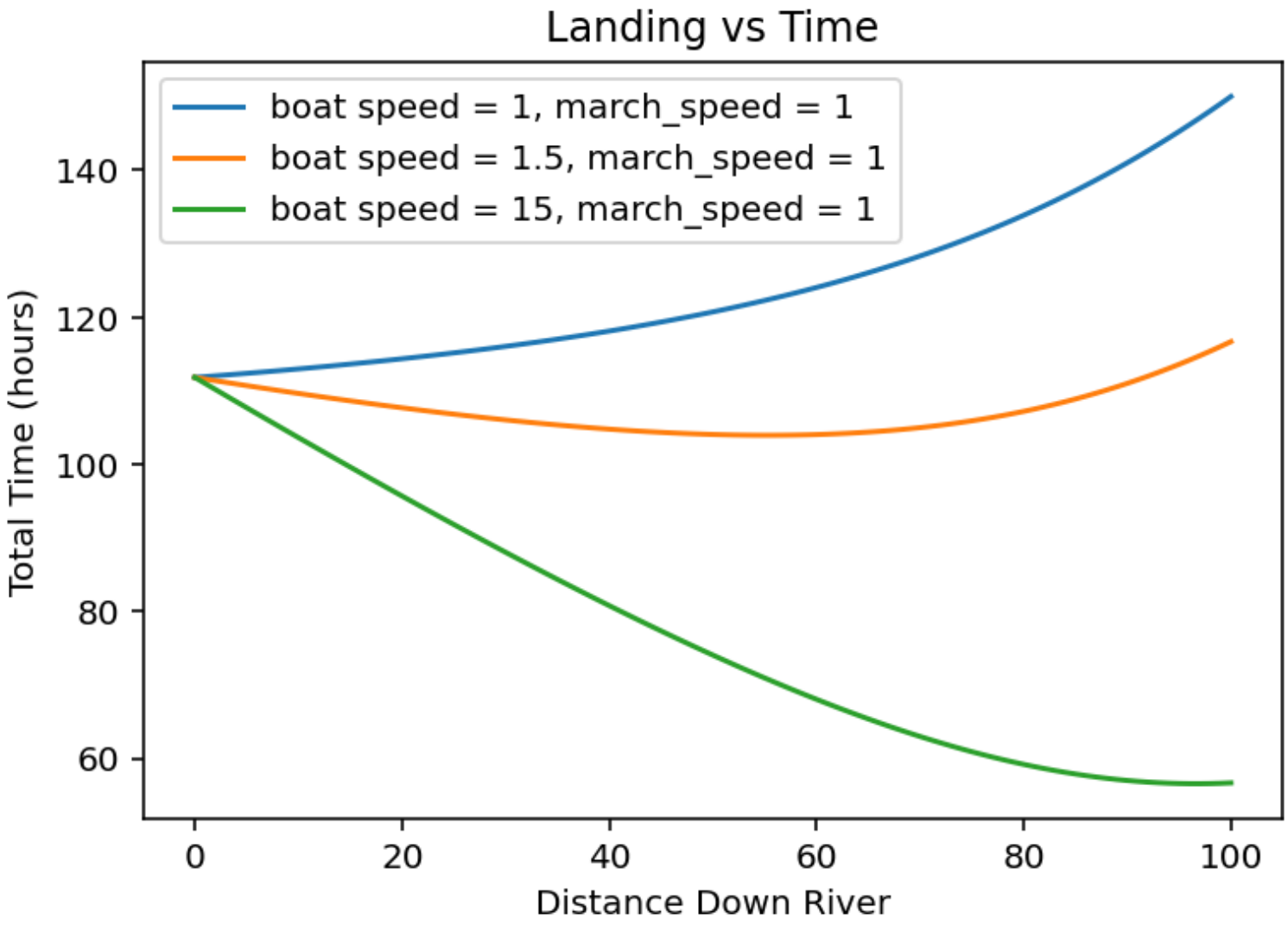
  Best landing is 55 miles south of start
  Distance marched would be 67.27 miles
  Time is 103.93 hours

- Let's try `find_landing(15, 1, 100, 50)`

  Best landing is 97 miles south of start
  Distance marched would be 50.09 miles
  Time is 56.56 hours

Optimization code allows us to play "what if"

Plot Provides More Information



Add a Constraint

- When marching, soldiers deserted at a rate of *1* soldier per mile marched
- The general did want to lose more than *55* soldiers to desertion

Let's Formalize It

$a + c$ = north-south distance

d = distance east

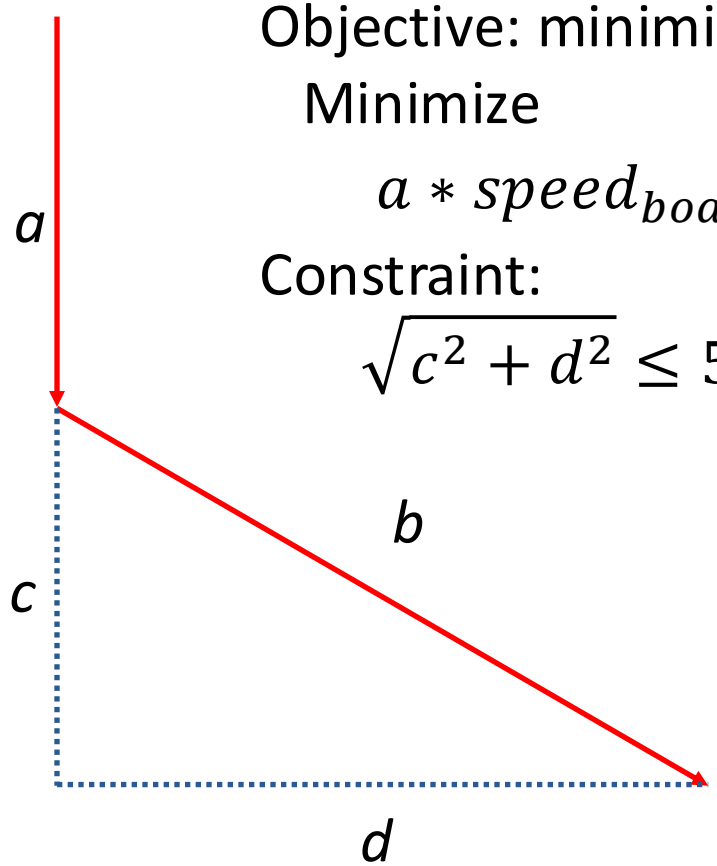
Objective: minimize

Minimize

$$a * speed_{boat} + \sqrt{c^2 + d^2} * speed_{march}$$

Constraint:

$$\sqrt{c^2 + d^2} \leq 55$$



New Code

```
def find_landing_1(boat_speed, march_speed, dist_south, dist_west,
                   des_rate, max_des):
    times = []
    incr = 1
    miles_south = 0
    max_march_dist = max_des*des_rate
    while miles_south <= dist_south:
        dist_marched = ((dist_south-miles_south)**2+dist_west**2)**0.5
        if not dist_marched > max_march_dist:
            tot_time = miles_south/boat_speed + dist_marched/march_speed
            times.append(tot_time)
        else:
            times.append(np.inf)
        miles_south += incr

    best_idx = np.argmin(times)
    landing = best_idx*incr
    print(f'Best landing is {landing} miles south of start')
    dist_marched = ((dist_south-landing)**2+dist_west**2)**0.5
    print(f'Distance marched would be {dist_marched:.2f} miles')
    print(f'Number of desertions is {round(dist_marched*des_rate)}')
    print(f'Time is {times[best_idx]:.2f} hours')
```

Why parameters?

Let's Compare

```
print('Result with no constraint')
find_landing(1.5, 1, 100, 50)
print('\nResult with maximum desertions = 100')
find_landing_1(1.5, 1, 100, 50, 1, 100)
print('\nResult with maximum desertions = 55')
find_landing_1(1.5, 1, 100, 50, 1, 55)
```

Result with no constraint
Best landing is 55 miles south of start
Distance marched would be 67.27 miles
Time is 103.93 hours

Result with maximum desertions = 100
Best landing is 55 miles south of start
Distance marched would be 67.27 miles
Number of desertions is 67
Time is 103.93 hours

Result with maximum desertions = 55
Best landing is 78 miles south of start
Distance marched would be 54.63 miles
Number of desertions is 55
Time is 106.63 hours

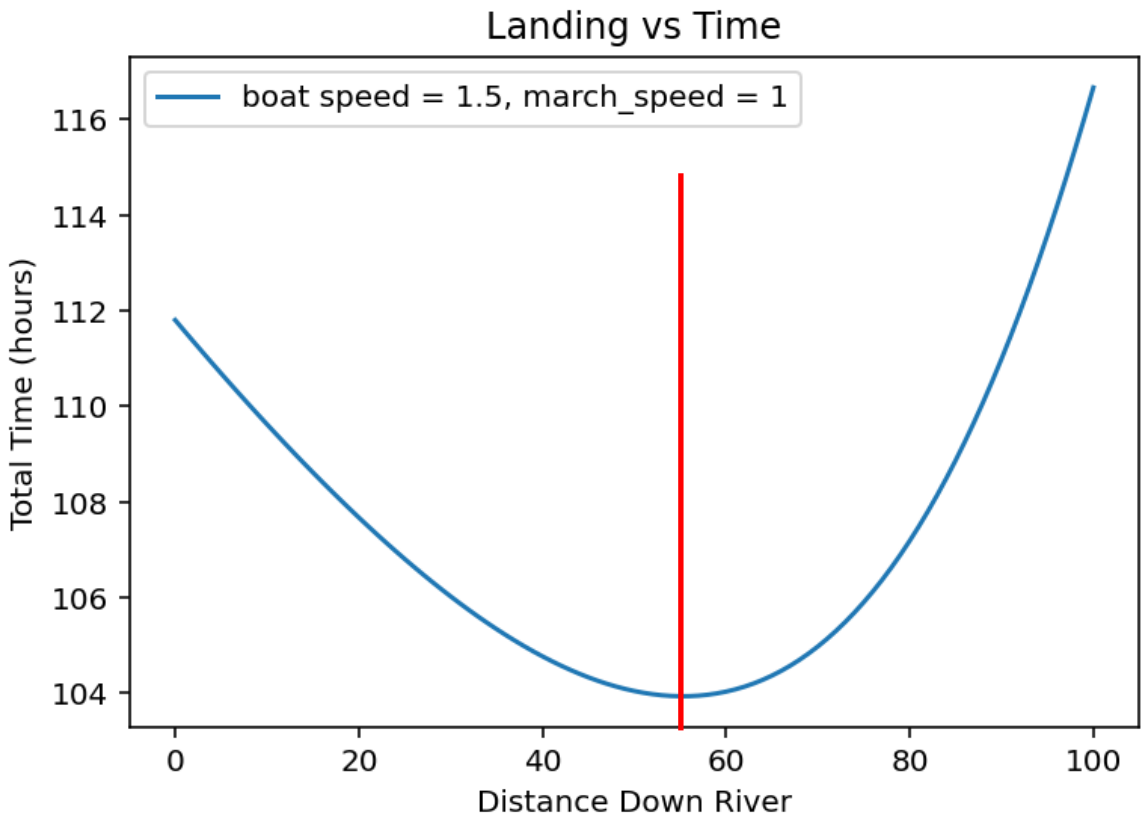
Suppose the Boat Could Land at Any Spot

- Problem is now continuous, not discrete, so can't enumerate all possible solutions
- However, problem is unimodal

Unimodal?

Unimodal

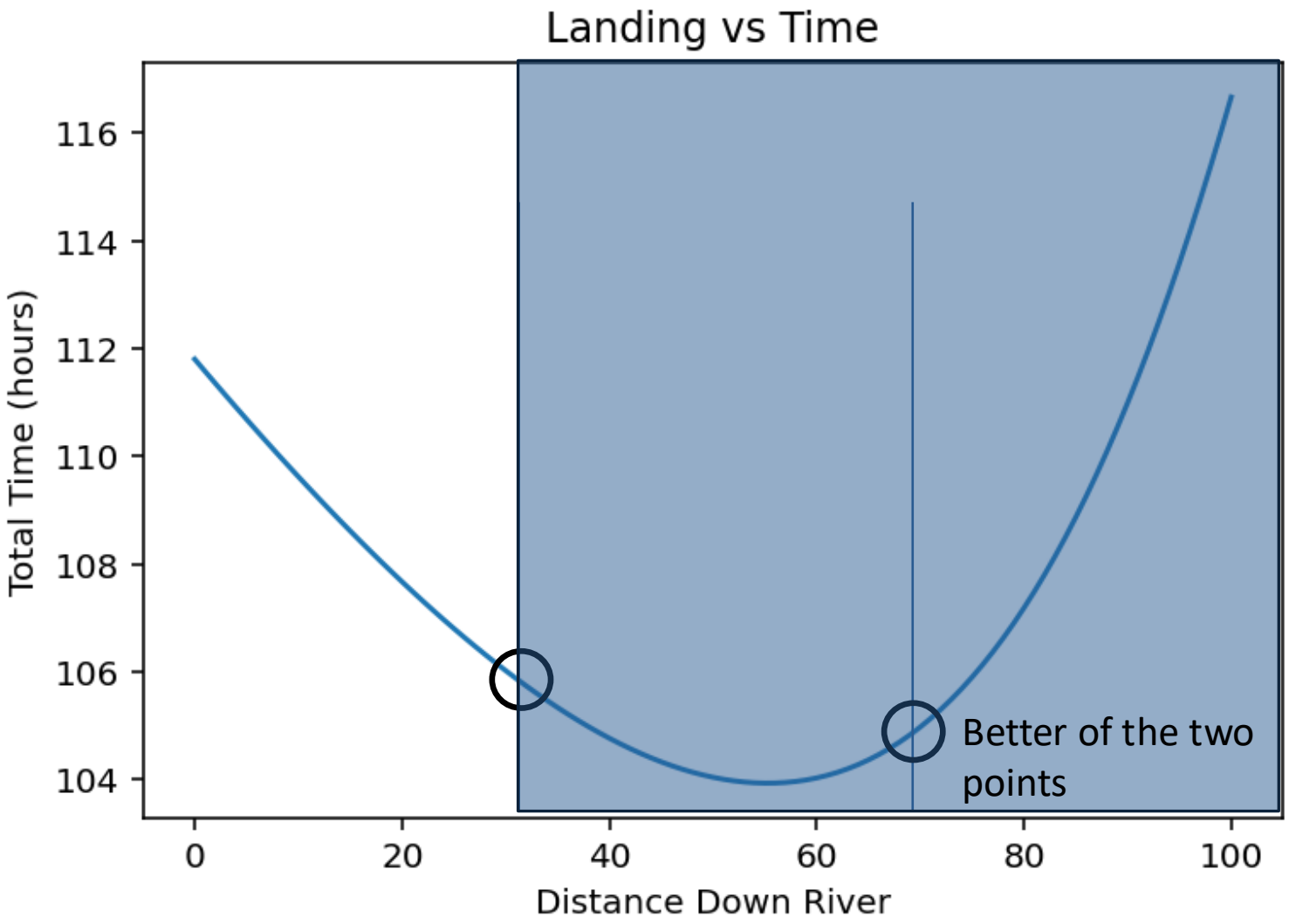
- One optimal time (at best landing place)
- Time strictly *decreasing before* that point
- Time strictly *increasing after* that point



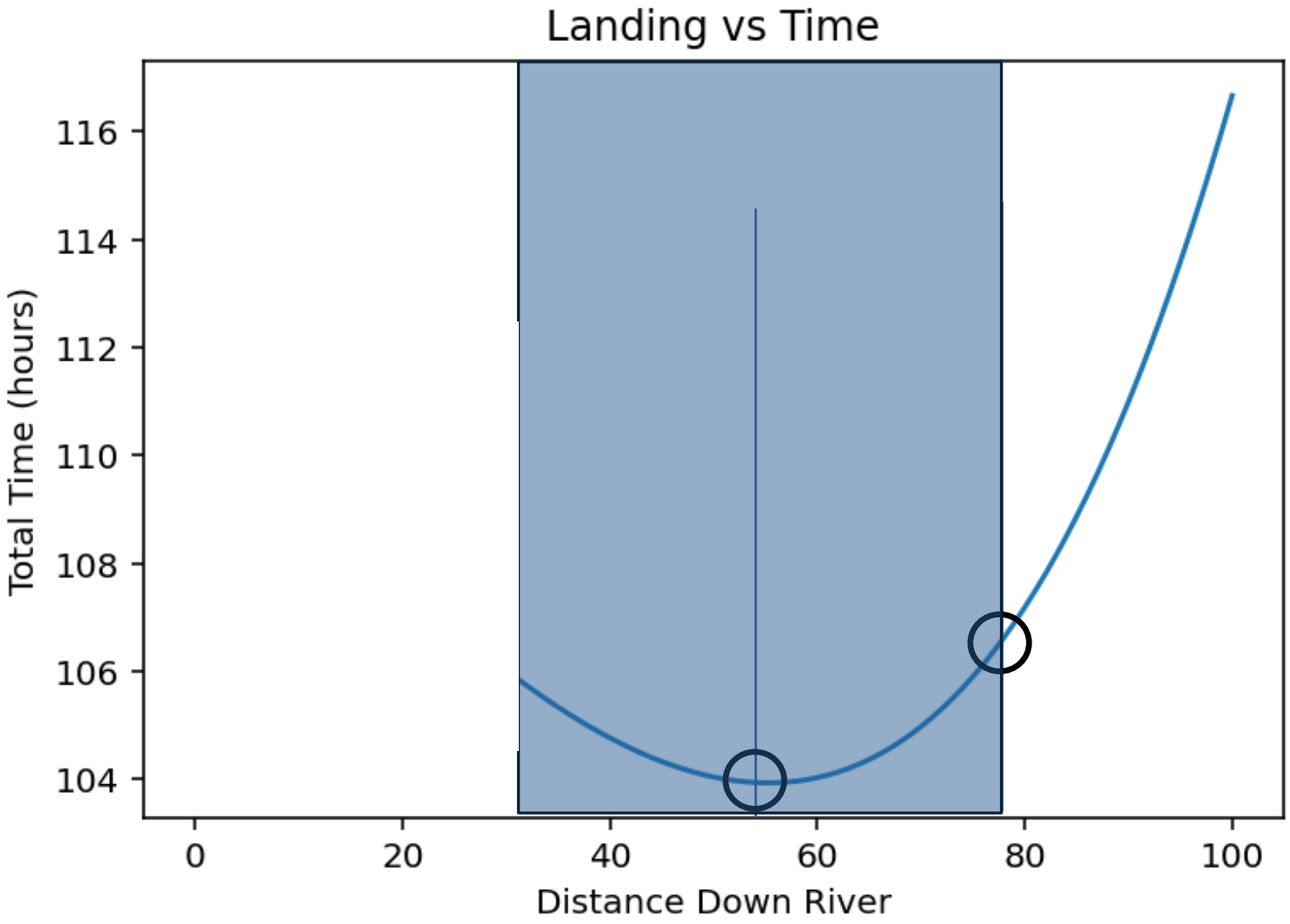
Ternary Search

- Want to minimize f(x) where x is in range (l, r)
- Like binary search except
  - Split interval into thirds
  - Discard the third that cannot contain the answer

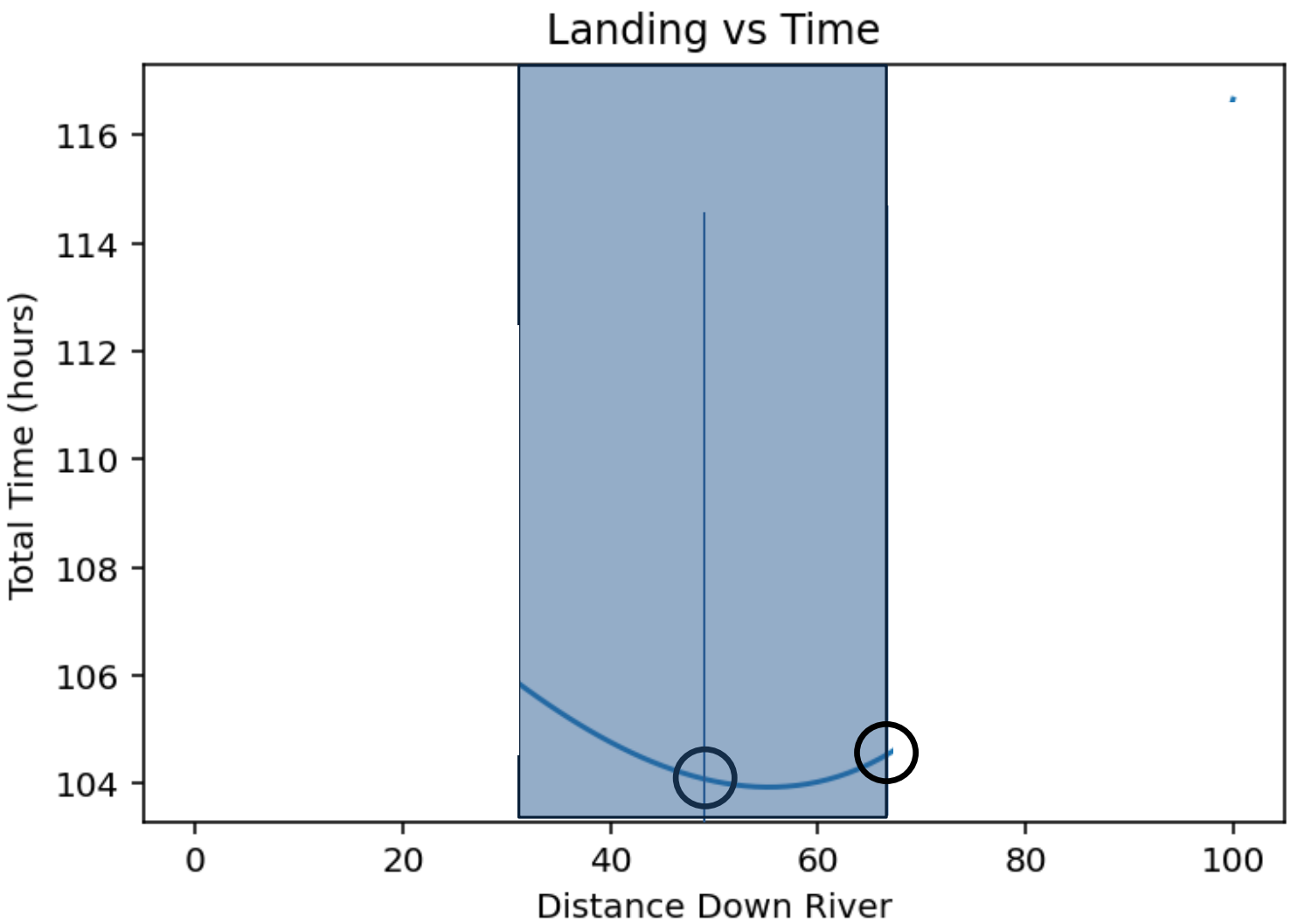
Visualizing Ternary Search



Visualizing Ternary Search



Visualizing Ternary Search



When the two lines are sufficiently close to each other

Take their average and call it a day

In Python

```python
def ternary_search(objective_fcn, min_val, max_val, precision):
    low, high = min_val, max_val
    # while high and low are too far apart
    while high - low > precision:
        guess1 = low + (high - low) / 3
        guess2 = high - (high - low) / 3
        print(f'min = {guess1:.4f}, max = {guess2:.4f}')
        if objective_fcn(guess1) > objective_fcn(guess2):
            low = guess1 # discard lower third
        else:
            high = guess2 # discard upper third
    return (low + high) / 2
```

General purpose function

Using Ternary Search

```
def time_taken(boat_speed, march_speed, dist_south, dist_east,
               landing):
    dist_marched = ((dist_south-landing)**2+dist_east**2)**0.5
    return landing/boat_speed + dist_marched/march_speed

objective_fcn = lambda landing: time_taken(1.5, 1, 100, 50,
                                           landing)
landing = ternary_search(objective_fcn, 0, 100, 0.01)
tot_time = objective_fcn(landing)
print(f'Best landing is {landing:.4f} miles south of start')
print(f'Time is {tot_time:.2f} hours')
```

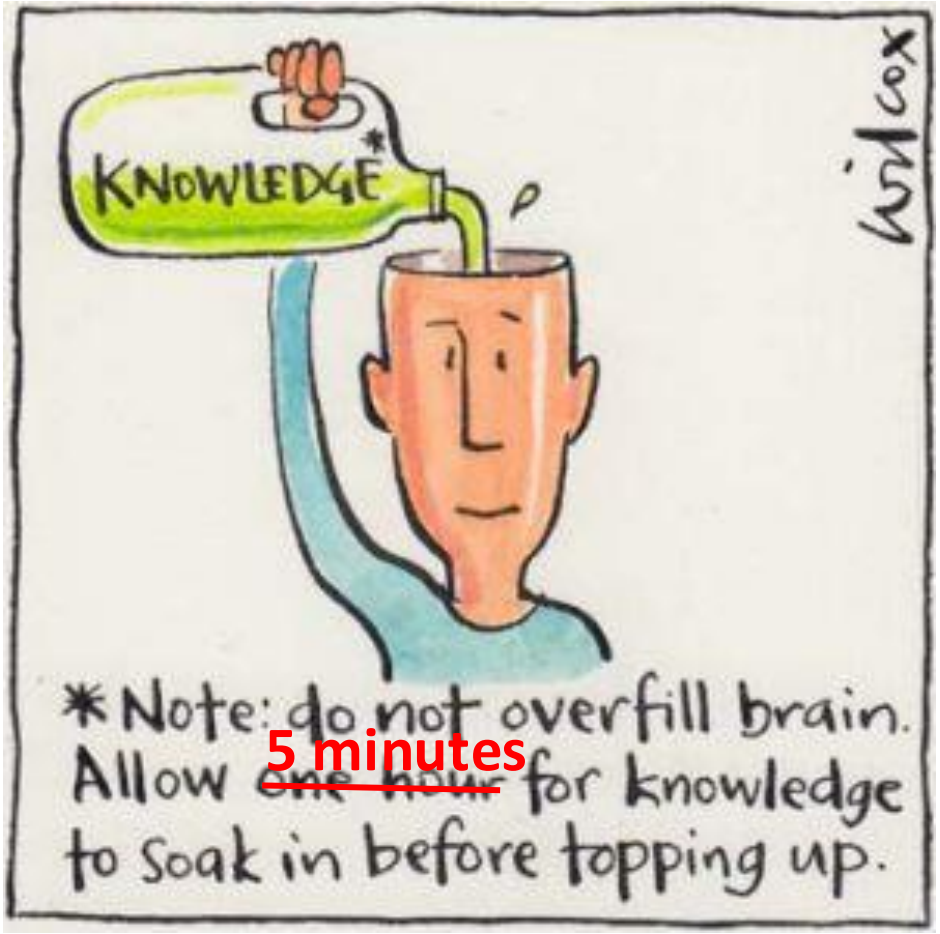
When We Run It

min = 33.3333, max = 66.6667
min = 55.5556, max = 77.7778
min = 48.1481, max = 62.9630
min = 43.2099, max = 53.0864
min = 49.7942, max = 56.3786
…
min = 55.2696, max = 55.2847
min = 55.2797, max = 55.2897
min = 55.2763, max = 55.2830
min = 55.2741, max = 55.2786
Best landing is 55.2786 miles south of start
Time is 103.93 hours

Summarizing to This Point

- Optimization problems cover a range of interesting scenarios
  - Often characterized by desire to maximize (or minimize) an objective function, subject to a constraint(s) on some parameters of problem
- Exhaustive enumeration can solve discrete problems
- Ternary search can solve continuous ones if they are unimodal
- Writing code to solve optimization problems can
  - Provide an answer to a specific question
  - Allow one to compare answers to related questions
- Next we will look at a different and important optimization problem

First, Five Minute Break



Imagine that You Are a Burglar

Disclaimer: We are not endorsing this career path!

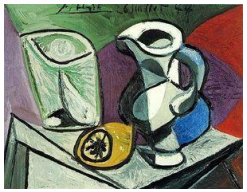
Knapsack Problems

- You have limited strength, so there is a maximum weight knapsack that you can carry
- You want to take more stuff than you can carry
- How do you choose what to take vs. leave behind?
  - Want to optimize "value" of things to take
- Two variants
  - Continuous or fractional knapsack problem — Straightforward
  - 0/1 knapsack problem — Much more interesting

Also known as a rucksack, haversack, backpack

Quanta are infinitesimal relative to available space

versus



Quanta are large relative to available space

Some Typical Applications

- Choosing teams in fantasy sports
- Capital budgeting
- Disaster relief shipments
- Allocating advertising budget

Stop and Think

- Is it a knapsack problem?
  - Packing for a vacation
  - Ranking professors by quality
  - Picking stocks for a portfolio
  - Finding the shortest path between two places
  - Debugging a Python program
  - Deciding which subjects to take freshmen year
  - Managing work/life balance

Stop and Think

- Is it a knapsack problem?
  - Packing for a vacation
  - Ranking professors by quality
  - Picking stocks for a portfolio
  - Finding the shortest path between two places
  - Debugging a Python program
  - Deciding which subjects to take freshmen year
  - Managing work/life balance

Our Running Example

- You are about to sit down to a meal
- You know how much you value different foods, e.g., you like donuts more than Brussels sprouts
- But you have a calorie budget, e.g., you don't want to consume more than 750 calories
- Choosing what to eat (assuming you are limited by calories, price, etc.) is a knapsack problem

NALD'S	BURGER KING	PIZZA HUT	KFC	HARV
:Nuggets	Double Whopper & cheese	BBQ Americano (14 inch)	Crispy Twister	Harvester & cheese
	963 calories	2,848 calories	510 calories	970 calori
	Whopper & cheese	Double Pepperoni (13 inch)	Fillet Burger	Scampi &
	721 calories	2,392 calories	441 calories	950 calori
d Egg	Chicken Royale	Margherita (13 inch)	Popcorn chicken (kids portion)	BBQ Chick Sandwich
	606 calories	2,256 calories	260 calories	950 calori
Sandwich	Regular fries	Double Pepperoni (9 inch)	Regular fries	Mixed grill
	300 calories	1,200 calories	260 calories	910 calori
	Hamburger	Margherita (9 inch)	Original recipe one chicken piece	Salmon fil beans
	275 calories	1,128 calories	258 calories	380 calori

For Example

You have a limit on consumed calories; would like to optimize choice of foods

How you measure "optimal" is your choice; we will focus on value (how much you enjoy a food)

0/1 Knapsack Problem, Formalized

- Each item is represented by a pair, *<value, weight>*
  - Referents for **value** and **weight (cost)** will vary by problem
  - For food, we will use **enjoyment** and **calories**
- The knapsack can accommodate items with a total weight of no more than *w*
- A vector, *A*, of length *n*, represents the set of available items. Each element of vector is an item
- A vector, *T*, of length *n*, is used to indicate whether or not items are taken. If *T[i] = 1*, item *A[i]* is taken. If *T[i] = 0*, item *A[i]* is not taken

Hence name 0/1 knapsack

For Example

Here is a set of items, each with a "value" (enjoyment) and a "weight" or "cost" (calories)

Food	wine	beer	pizza	burger	fries	coke	apple	donut	cake
Value	89	90	30	50	90	79	90	10	85
Cals	123	154	258	354	365	150	95	195	107

A = [, , , , , , , ,]

If we select a vector T, for example:

T = [0 , 1 , 0 , 1 , 0 , 0 , 0 , 0 , 1]

that means we are selecting the subset:

beer, burger, cake

0/1 Knapsack Problem, Formalized

Find a T that maximizes

$$\sum_{i=0}^{n-1} T[i] \times A[i].\text{value}$$

subject to the constraint that

$$\sum_{i=0}^{n-1} T[i] \times A[i].\text{weight} \leq w$$

Going from an informal understanding of a problem to a rigorous problem statement is an important skill to develop.

Vague problem → Rigorous problem → Algorithm → Code

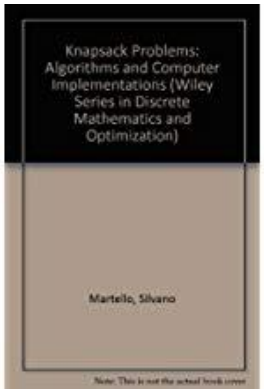
statement statement

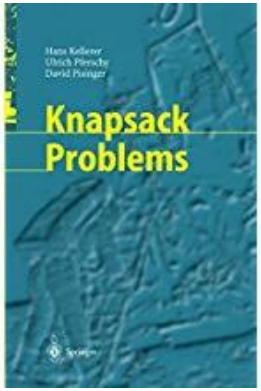
Real World
(6.100B lectures)

6.100A

Many Closely Related Problems

- Bin packing problem
  - Items of different volumes must be packed into a finite number of bins, each of a fixed given volume, in a way that minimizes the number of bins used
- Multiple knapsack problem
  - Like Bin Packing Problem, but only a subset of items may be selected; in Bin Packing Problem, all items have to be packed into certain bins
- Integer knapsack problem
- Multiple constraint knapsack problem
- …





Solving the 0/1 Knapsack Problem: **BRUTE FORCE**

Brute Force Strategy

1. Enumerate **all possible combinations** of items. That is to say, generate all subsets of the set of items. This is called the **power set**.
2. Remove all of the combinations whose total units exceeds the allowed weight.
3. From the remaining combinations **choose any one whose value is the largest**.

This works because it is a discrete problem

Problem Input: A menu

Food	wine	beer	pizza	burger	fries	coke	apple	donut	cake
Value	89	90	30	50	90	79	90	10	85
Cals	123	154	258	354	365	150	95	195	107

- Let's consider how to represent this information in Python

Problem Input: Class Item

```
class Item(object):
    def __init__(self, n, v, w):
        self._name = n
        self._value = v
        self._cost = w
    def get_value(self):
        return self._value
    def get_cost(self):
        return self._cost
    def get_density(self):
        if self._cost > 0:
            return self._value / self._cost
        else:
            return float('inf')
    def __str__(self):
        return (f'{self._name}: '
                f'<{self._value}, {self._cost}>')
```

Initial underscore indicates "internal" attribute

Getter methods

Problem Input: A Menu of Foods

```python
names = ['wine', 'beer', 'pizza', 'burger',
         'fries', 'cola', 'apple', 'donut', 'cake']
values = [89, 90, 95, 100, 90, 79, 50, 10, 85]
calories = [123, 154, 258, 354, 365, 150, 95, 195, 107]

def build_menu(names, values, calories):
    """names, values, calories are lists of same length
       names are strings
       values and calories are non-negative numbers
       returns a list of Items"""
    menu = []
    for i in range(len(values)):
        menu.append(Item(names[i], values[i], calories[i]))
    return menu

calorie_limit = 750
```

Brute Force Solution

- Generate all possible combinations of items
  - Represented as binary vectors
- Chose one that has highest value while honoring constraints

Generating the Potential Solutions

```
def generate_combinations(n):
    """Assumes n is a non-negative integer.
    Returns a list containing all binary strings of length n."""
    if n == 0:
        return []
    else:
        return [format(i, f'0{n}b') for i in range(2**n)]
```

?

Generating the Potential Solutions

```
def generate_combinations(n):
    """Assumes n is a non-negative integer.
    Returns a list containing all binary strings of length n."""
    if n == 0:
        return []
    else:
        return [format(i, f'0{n}b') for i in range(2**n)]
```

Converts each int to a string

Generates sequence of ints

b means binary

0{n} means pad with zeros to get binary string of length n.

Let's Test It

```python
def test_combinations():
    test_cases = [0, 1, 2, 3]
    expected_results = [[], ['0', '1'], ['00', '01', '10', '11'],
        ['000', '001', '010', '011', '100', '101', '110', '111']]

    for n, expected in zip(test_cases, expected_results):
        actual = generate_combinations(n)
        if actual == expected:
            print(f"Test for n = {n} passed: {actual}")
        else:
            print(f'Test for n = {n} failed.',
                  f'Expected: {expected}, Actual: {actual}')

test_combinations()
```

```
Test for n = 0 passed: []
Test for n = 1 passed: ['0', '1']
Test for n = 2 passed: ['00', '01', '10', '11']
Test for n = 3 passed: ['000', '001', '010', '011', '100',
'101', '110', '111']
```

Enough About the Subroutine

- Let’s use it to solve 0/1 knapsack problem

The Code

```
def brute_force(items, cost_limit):
    """items a list, cost_limit >= 0
       solves the 0/1 knapsack problem with items and cost_limit
       returns as a list a subset of items that don't exceed cost_limit
       while maximizing total value, and also the value of that list"""
    n = len(items)
    all_combinations = generate_combinations(n)
    best_value = 0
    best_subset = []
    for combination in all_combinations:
        subset = [items[i] for i in range(n) if combination[i] == '1']
        subset_cost = sum([item.get_cost() for item in subset])
        if subset_cost <= cost_limit:
            subset_value = sum([item.get_value() for item in subset])
            if subset_value > best_value:
                best_value = subset_value
                best_subset = subset
    return best_subset, best_value
```

Test it out

```
names = ['wine', 'beer', 'pizza', 'burger', 'fries',
         'cola', 'apple', 'donut', 'cake']
values = [89, 90, 95, 100, 90, 79, 50, 10, 85]
calories = [123, 154, 258, 354, 365, 150, 95, 195, 107]

calorie_limit = 750
foods = build_menu(names, values, calories)
solution, value = brute_force(foods, calorie_limit)
print(f'Total value of items taken = {value}')
for item in solution:
    print(f'   {item}')
```

```
Total value of items taken = 409
   wine: <89, 123>
   beer: <90, 154>
   pizza: <95, 258>
   apple: <50, 95>
   cake: <85, 107>
```

wine	5
beer	3
pizza	2
apple	8
cake	6

Rank by value

Note that best solution does not include most valuable item

Summarizing

- Optimization problems cover a range of interesting scenarios
  - Often characterized by desire to maximize (or minimize) an objective function, subject to a constraint(s) on some parameters of problem
- Can be discrete or continuous
  - Different algorithms needed
- Writing code to solve optimization problems can
  - Provide an answer to a specific question
  - Allow one to compare answers to related questions
- 0/1 knapsack problem describes one broad class of optimization problems
  - Brute force algorithm worked on a small example
- Next week we will look at larger problems